

6.6

Improving Convergence with Batch Normalization

Part 1: Scaling Layer Inputs Using BatchNorm

Sebastian Raschka and the Lightning AI Team

Batch normalization (BatchNorm)
can improve convergence and
reduce overfitting

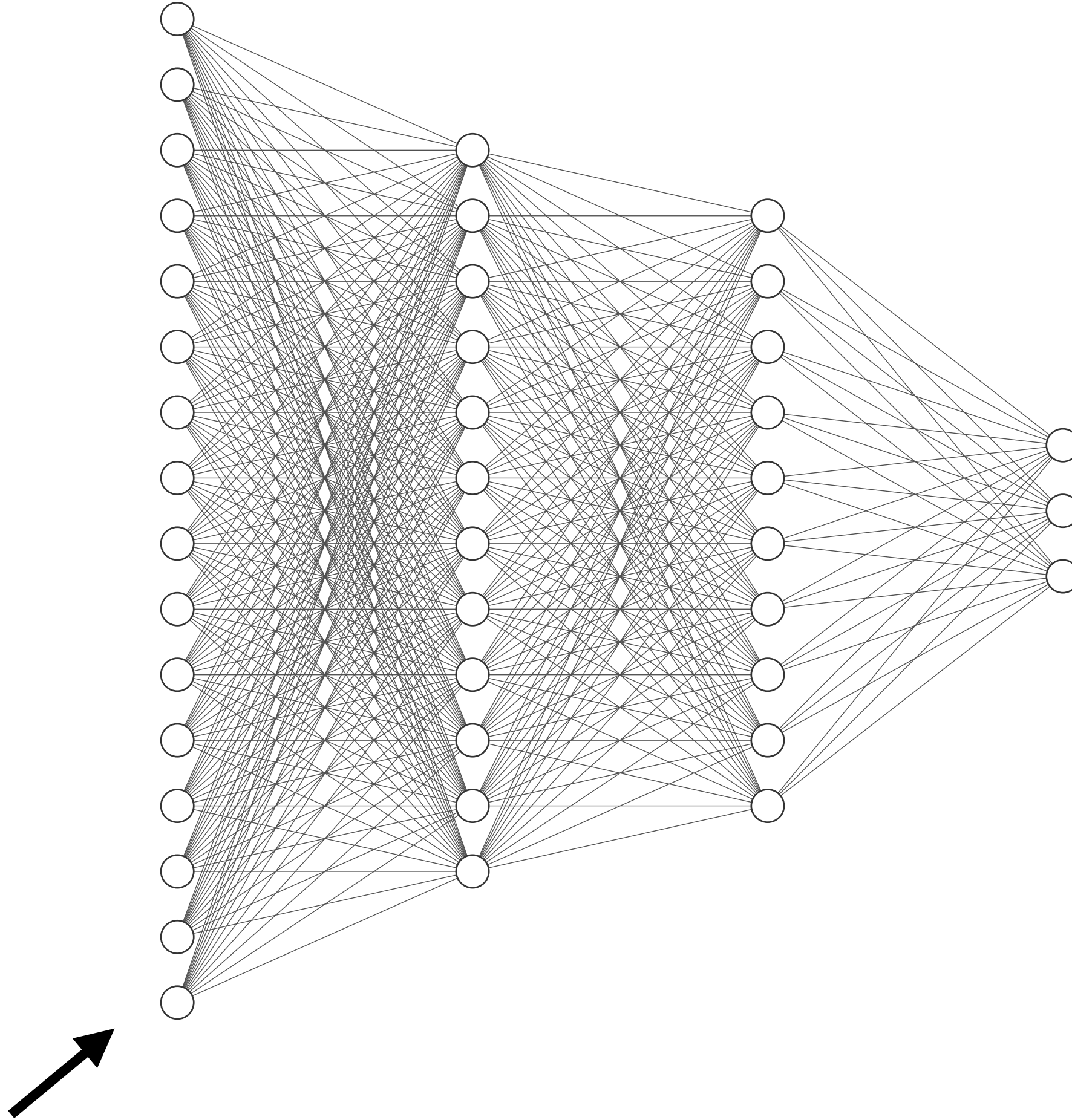
Ioffe & Szegedy (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. <https://arxiv.org/abs/1502.03167>

Sebastian Raschka

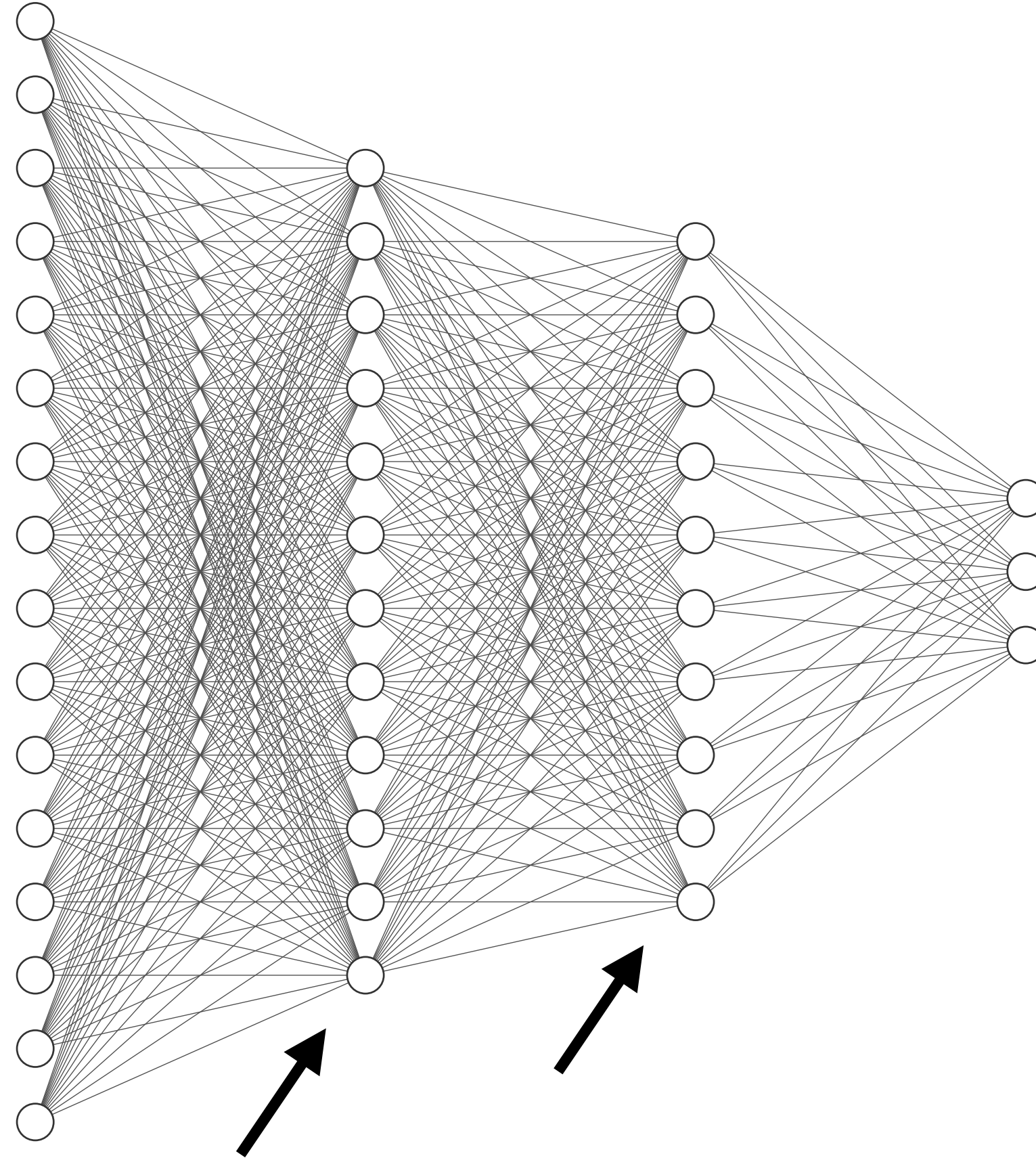
Deep Learning Fundamentals, Unit 6

Lightning AI

Remember when we talked about
standardizing the input **features**?



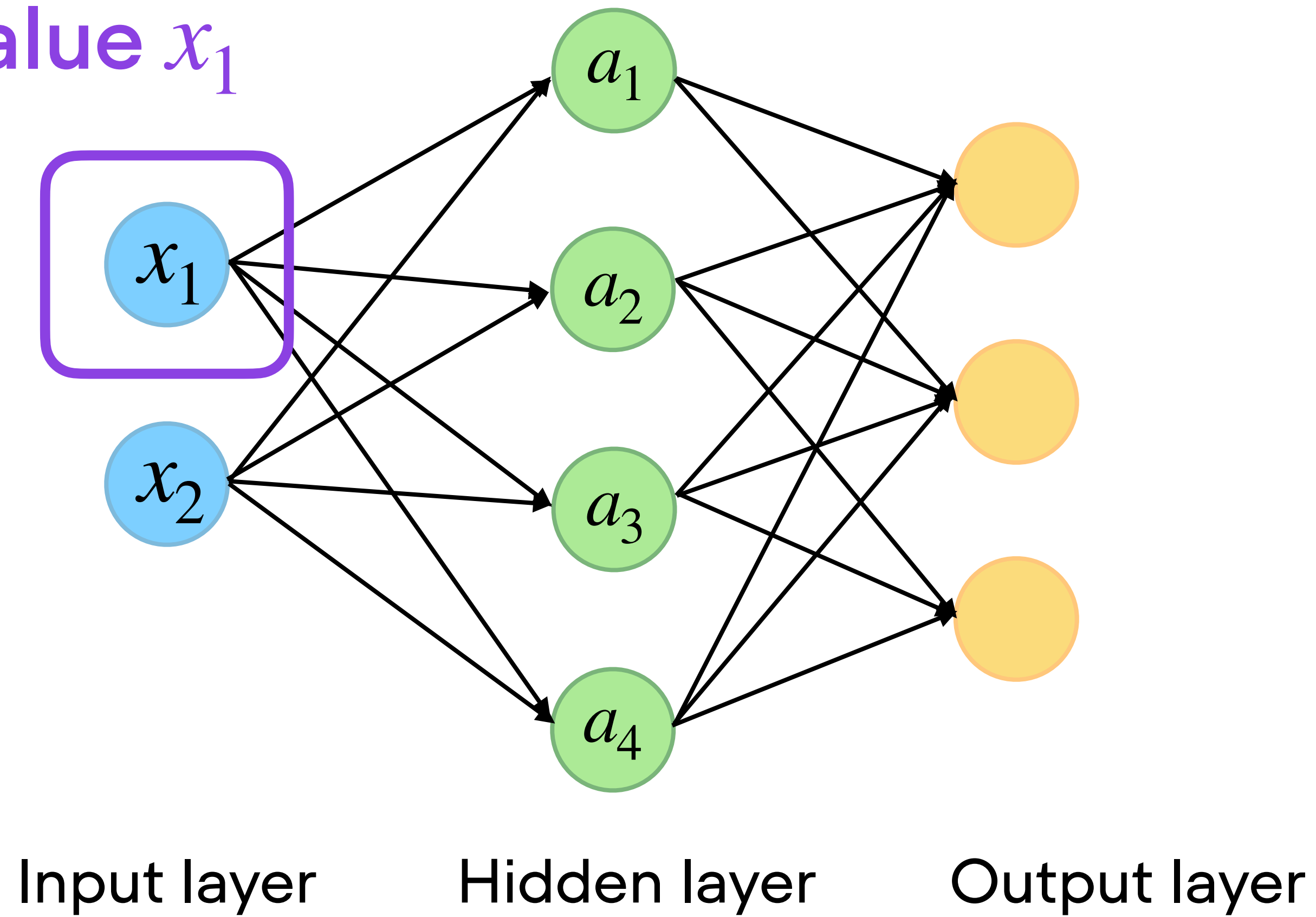
We standardized the **input features**



Using BatchNorm, we additionally "standardize" data at the **hidden layers**

How did we **standardize** again?

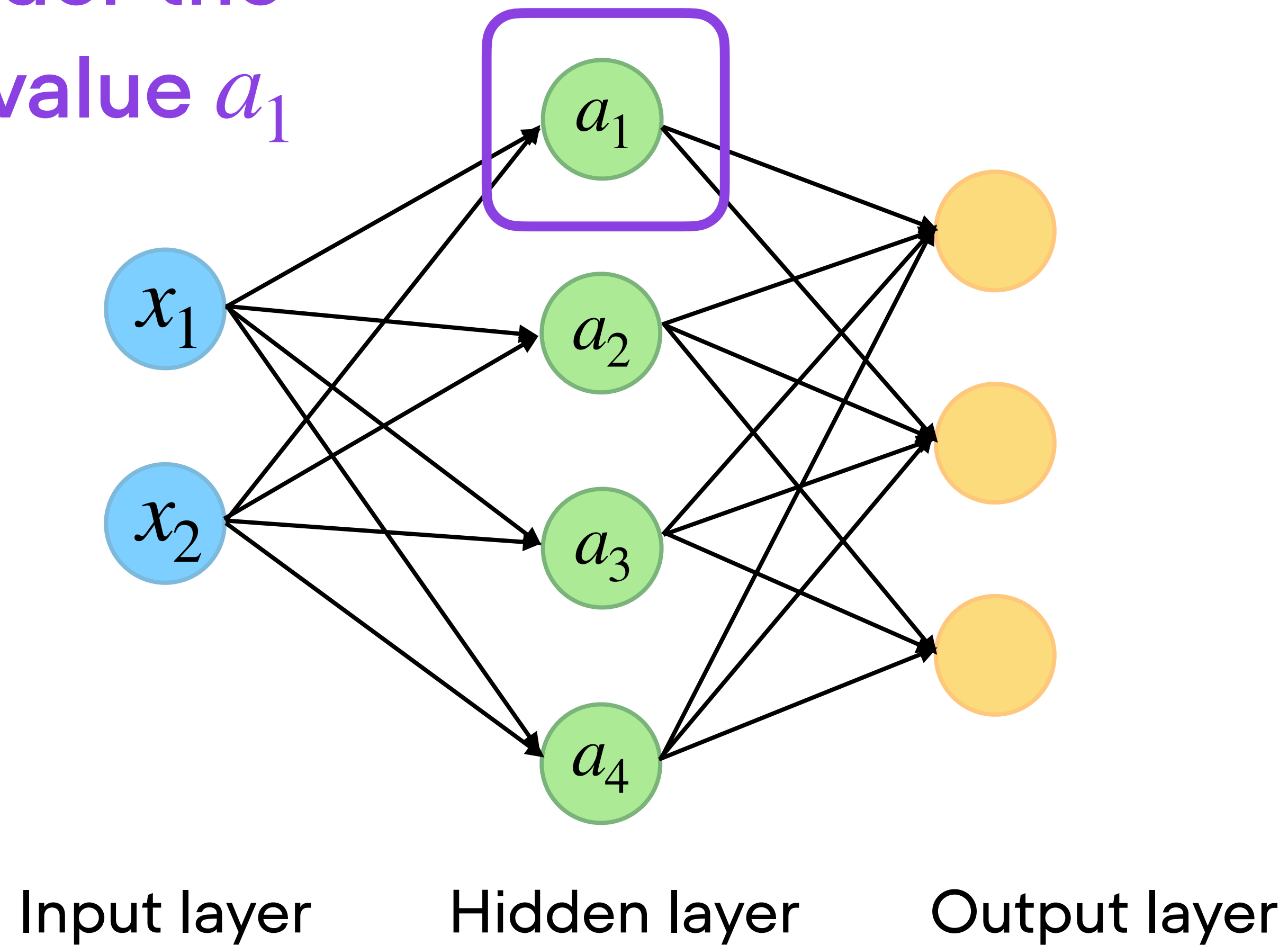
Consider the input
feature value x_1



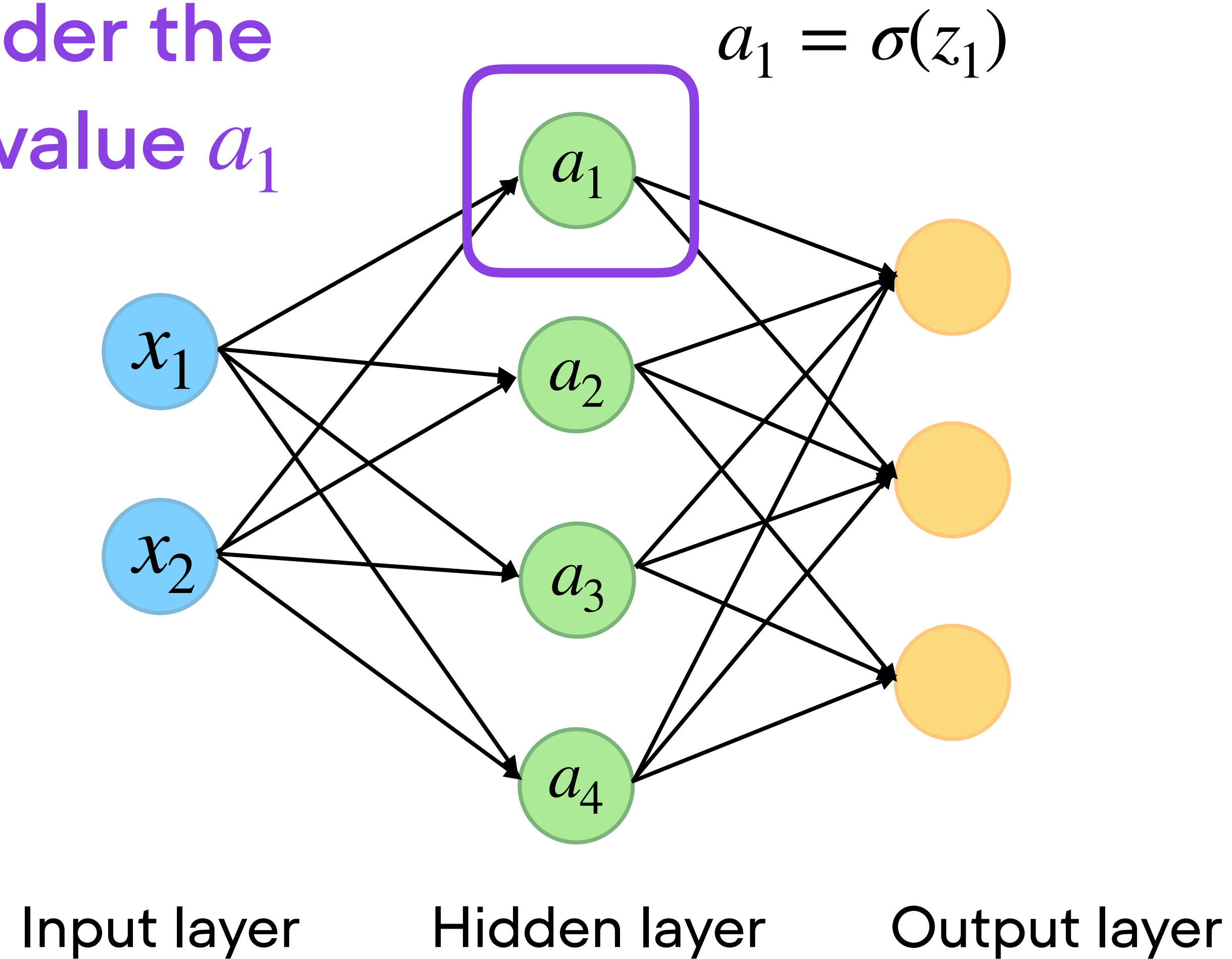
Feature standardization (Unit 1.6)

$$x_1'^{[i]} = \frac{x_1^{[i]} - \text{mean}(x_1)}{\text{std}(x_1)}$$

Now consider the
activation value a_1



Now consider the
activation value a_1



We use BatchNorm to **normalize hidden layer activations**

Step 1: Standardize net inputs

$$z_1'^{[i]} = \frac{z_1^{[i]} - \text{mean}(z_1)}{\text{std}(z_1) + \epsilon}$$

 small number to avoid division by 0

Step 2: Pre-activation scaling

$$a_1'[i] = \gamma_1 \cdot z_1'[i] + \beta_1$$

controls the spread

controls the mean

Step 2: Pre-activation scaling

$$a_1'[i] = \gamma_1 \cdot z_1'[i] + \beta_1$$


learnable parameters

Step 2: Pre-activation scaling

$$a_1'[i] = \gamma_1 \cdot z_1'[i] + \beta_1$$

if this is 1

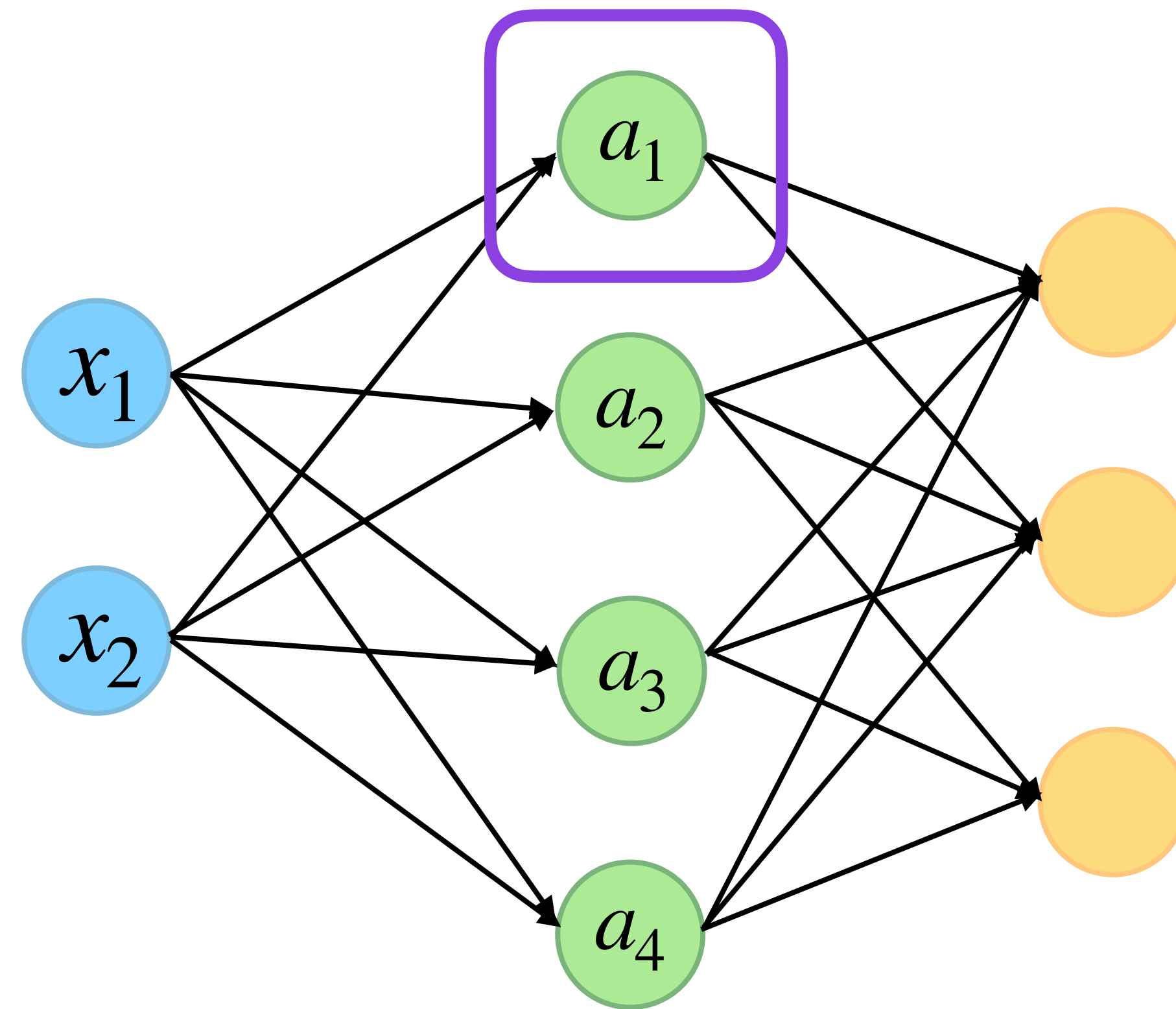
if this is 0

then steps 1 & 2 equal standardization

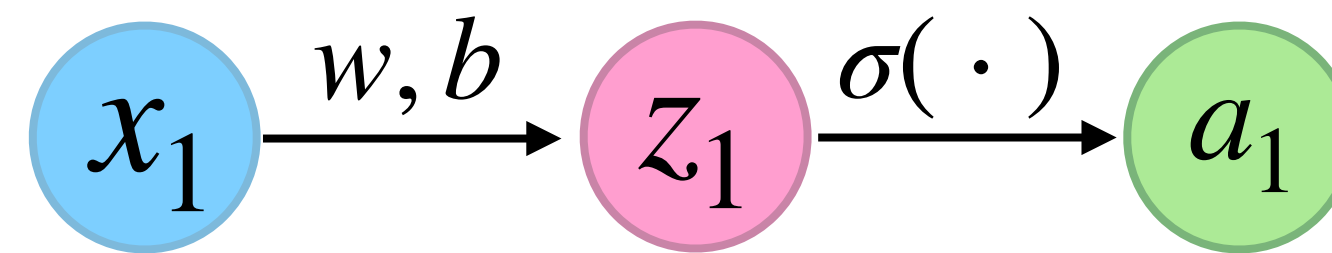
Step 3: Compute activation

$$a_1 = \sigma(a_1'^{[i]})$$

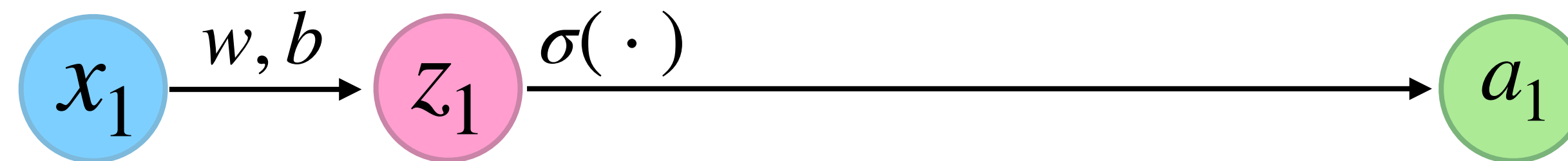
Regular hidden layer activation without BatchNorm



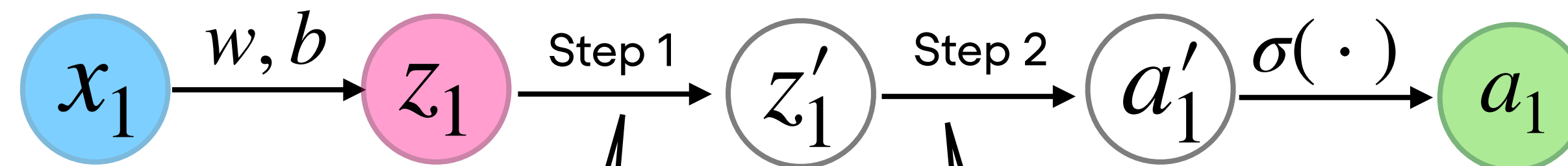
Regular hidden layer activation without BatchNorm



Without BatchNorm



With BatchNorm



$$z_1^{[i]} = \frac{z_1^{[i]} - \text{mean}(z_1)}{\text{std}(z_1) + \epsilon}$$

$$a_1^{[i]} = \gamma_1 \cdot z_1^{[i]} + \beta_1$$

NEXT: BatchNorm in PyTorch